RA-001 Knowledge Store on AWS

The reference architecture: Knowledge Store as one Terraform module on AWS

Reversed from the implementation at 0.3.0. Draft.

Context

One AWS account, one Terraform module, two applies. Ingestion as a Fargate sweep started by uploads; the record in S3; the graph in Neptune; the passage index on S3 Vectors; the tools behind AgentCore Gateway; the agent on AgentCore Runtime with Bedrock models and Guardrails; the page and API on CloudFront, API Gateway, Lambda and DynamoDB; sign-in with Cognito or a host website; images built in the account.

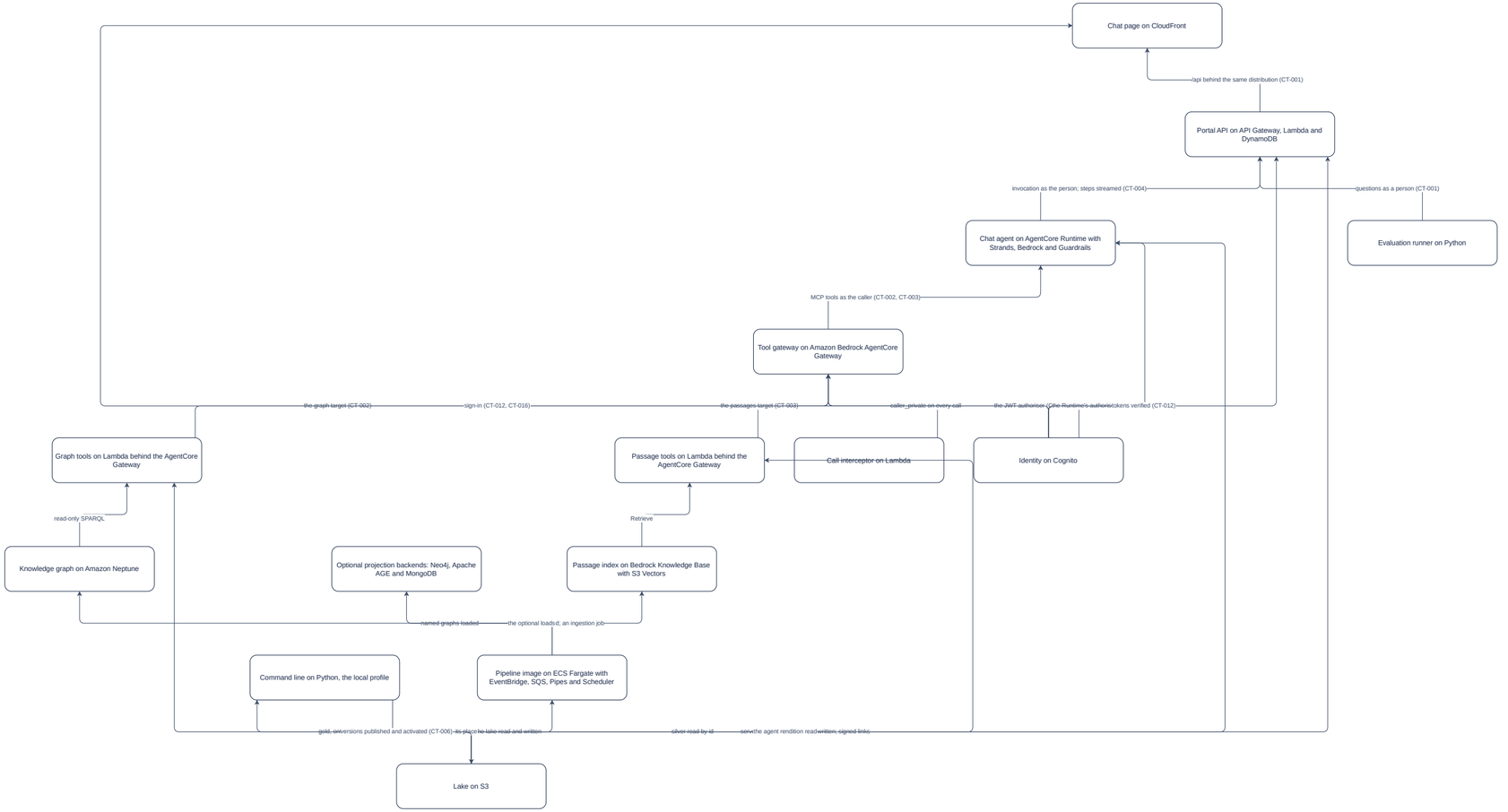
Non-goals

Multi-region, multi-account and a shared tenant model are out of scope: one deployment is one account and one stack.

Non-goals

Multi-region, multi-account and a shared tenant model are out of scope: one deployment is one account and one stack.

Diagram



Generated from the Required Building Blocks and Interfaces tables with the model skill (`components.drawio` is the source). The illustrated reference architecture is aws/architecture.svg, from aws/architecture.drawio and its generator architecture.py.

Required Building Blocks (1 of 2)

Building Block	Realises	Role
SBB-001 Lake on S3	ABB-002	The record
SBB-002 Pipeline image on ECS Fargate with EventBridge, SQS, Pipes and Scheduler	ABB-024	The sweep: every stage in one image, started by uploads and a schedule
SBB-003 Knowledge graph on Amazon Neptune	ABB-011	The SPARQL projection
SBB-004 Passage index on Bedrock Knowledge Base with S3 Vectors	ABB-013	Search by meaning
SBB-005 Graph tools on Lambda behind the AgentCore Gateway	ABB-012	The graph and structured tools
SBB-015 Passage tools on Lambda behind the AgentCore Gateway	ABB-025	Search and read of passages
SBB-016 Tool gateway on Amazon Bedrock AgentCore Gateway	ABB-026	One MCP surface with the authoriser and the interception point
SBB-017 Call interceptor on Lambda	ABB-027	The scope decision

Required Building Blocks (2 of 2)

Building Block	Realises	Role
SBB-006 Chat agent on AgentCore Runtime with Strands, Bedrock and Guardrails	ABB-014	Answering
SBB-007 Portal API on API Gateway, Lambda and DynamoDB	ABB-019	The API, the quota and the chat state
SBB-018 Chat page on CloudFront	ABB-018	The page
SBB-008 Identity on Cognito	ABB-020	People and tokens
SBB-011 Optional projection backends: Neo4j, Apache AGE and MongoDB	ABB-011	When Neptune is not the graph
SBB-013 Command line on Python, the local profile	ABB-028	The store on one machine; the release commands
SBB-019 Evaluation runner on Python	ABB-021	Right, not only grounded

Every box is one container, as the calibration standard requires of a reference architecture. The image build (DN-003, DN-013), the Terraform module and the launch stack (CT-010) and the models (DN-011) are nodes, contracts and external systems, not boxes; the deployment view below places the services on the nodes.

Interfaces (1 of 2)

Interface	Provider	Consumer	Purpose
IF-01	SBB-001 Lake on S3	SBB-002 Pipeline image on ECS Fargate with EventBridge, SQS, Pipes and Scheduler	upload events (CT-005); the lake read and written
IF-02	SBB-002 Pipeline image on ECS Fargate with EventBridge, SQS, Pipes and Scheduler	SBB-003 Knowledge graph on Amazon Neptune	named graphs loaded
IF-03	SBB-002 Pipeline image on ECS Fargate with EventBridge, SQS, Pipes and Scheduler	SBB-004 Passage index on Bedrock Knowledge Base with S3 Vectors	passages synced; an ingestion job
IF-04	SBB-002 Pipeline image on ECS Fargate with EventBridge, SQS, Pipes and Scheduler	SBB-011 Optional projection backends: Neo4j, Apache AGE and MongoDB	the optional loads
IF-05	SBB-003 Knowledge graph on Amazon Neptune	SBB-005 Graph tools on Lambda behind the AgentCore Gateway	read-only SPARQL
IF-06	SBB-001 Lake on S3	SBB-005 Graph tools on Lambda behind the AgentCore Gateway	gold, ontology and config read
IF-07	SBB-004 Passage index on Bedrock Knowledge Base with S3 Vectors	SBB-015 Passage tools on Lambda behind the AgentCore Gateway	Retrieve
IF-08	SBB-001 Lake on S3	SBB-015 Passage tools on Lambda behind the AgentCore Gateway	silver read by id
IF-09	SBB-005 Graph tools on Lambda behind the AgentCore Gateway	SBB-016 Tool gateway on Amazon Bedrock AgentCore Gateway	the graph target (CT-002)
IF-10	SBB-015 Passage tools on Lambda behind the AgentCore Gateway	SBB-016 Tool gateway on Amazon Bedrock AgentCore Gateway	the passages target (CT-003)
IF-11	SBB-017 Call interceptor on Lambda	SBB-016 Tool gateway on Amazon Bedrock AgentCore Gateway	caller_private on every call
IF-12	SBB-008 Identity on Cognito	SBB-016 Tool gateway on Amazon Bedrock AgentCore Gateway	the JWT authoriser (CT-012)

Interfaces (2 of 2)

Interface	Provider	Consumer	Purpose
IF-13	SBB-016 Tool gateway on Amazon Bedrock AgentCore Gateway	SBB-006 Chat agent on AgentCore Runtime with Strands, Bedrock and Guardrails	MCP tools as the caller (CT-002, CT-003)
IF-14	SBB-008 Identity on Cognito	SBB-006 Chat agent on AgentCore Runtime with Strands, Bedrock and Guardrails	the Runtime's authoriser (CT-012)
IF-15	SBB-006 Chat agent on AgentCore Runtime with Strands, Bedrock and Guardrails	SBB-007 Portal API on API Gateway, Lambda and DynamoDB	invocation as the person; steps streamed (CT-004)
IF-16	SBB-008 Identity on Cognito	SBB-007 Portal API on API Gateway, Lambda and DynamoDB	tokens verified (CT-012)
IF-17	SBB-008 Identity on Cognito	SBB-018 Chat page on CloudFront	sign-in (CT-012, CT-016)
IF-18	SBB-007 Portal API on API Gateway, Lambda and DynamoDB	SBB-018 Chat page on CloudFront	/api behind the same distribution (CT-001)
IF-19	SBB-001 Lake on S3	SBB-007 Portal API on API Gateway, Lambda and DynamoDB	served layers read; requests written; signed links
IF-20	SBB-001 Lake on S3	SBB-006 Chat agent on AgentCore Runtime with Strands, Bedrock and Guardrails	the agent rendition read
IF-21	SBB-001 Lake on S3	SBB-013 Command line on Python, the local profile	the bucket, or a folder in its place
IF-22	SBB-013 Command line on Python, the local profile	SBB-001 Lake on S3	versions published and activated (CT-006)
IF-23	SBB-019 Evaluation runner on Python	SBB-007 Portal API on API Gateway, Lambda and DynamoDB	questions as a person (CT-001)

The contracts register, ../../contracts.csv, gives each interface its definition and versioning.

Deployment view

Service	Node	Store
SV-001 pipeline	DN-004 Fargate in DN-002	writes ST-002 to ST-006, ST-008, ST-009, ST-010, ST-015, ST-016
SV-002 chat-agent	DN-005 AgentCore Runtime	ST-011 agent# keys
SV-003 portal-api	DN-007 Lambda behind DN-008 and DN-009	ST-011; reads ST-004, ST-005, ST-003, ST-002
SV-004 tools-graph	DN-007 Lambda in DN-002 behind DN-006	reads ST-008, ST-004, ST-005
SV-005 tools-passages	DN-007 Lambda behind DN-006	reads ST-007, ST-003, ST-004
SV-006 agent-interceptor	DN-007 Lambda on DN-006	none
SV-007 chat-page	DN-009 CloudFront from ST-012	browser storage

Controls

Those of AP-001, plus: the origin secret between CloudFront and the API; reserved concurrency and the API throttle; the web ACL input; IAM authentication and private subnets for Neptune; a role per unit with a permissions boundary; the budget alert; images pinned by digest. The full table is security.md.

Cost

The cost table in aws/aws.md is the cost model: the Neptune instance is the fixed cost; everything else is per use; model calls are the cost that matters.

Decisions

ADR-006 one module built in-account; ADR-007 the passage index on S3 Vectors; ADR-008 memory by default with optional backends; ADR-009 tables bound as snapshots. Open: CN-001 the graph at scale, CN-006 the deployables along the contexts.